PYTHON

Stop Learning All of Pandas. These 12 Functions Are Enough

The 12 Pandas functions that cover almost all data analyst work: load, inspect, clean, filter, summarise, join and reshape. Each with real code.

INSIDE THIS PDF · 8 PAGES

- **Load and inspect: 1 to 3**
- **Clean: 4 to 6**
- **Filter: 7 and 8**
- **Summarise: 9 and 10**
- **Join and reshape: 11 and 12**
- **All 12 in one analysis**
- **What to skip, and why**

@thedataguy16

Load and inspect: 1 to 3

Run these on every new dataset before any analysis.

1. read_csv

Load the file. Set types and dates up front to avoid problems later.

```
df = pd.read_csv("sales.csv",
    parse_dates=["order_date"],
    dtype={"pincode": str})
```

2. info

Rows, columns, types and non-null counts in one view. Spot numbers stored as text.

```
df.info()
```

3. isna

Count missing values per column.

```
df.isna().sum().sort_values(ascending=False)
```

@thedataguy16

Clean: 4 to 6

Most analysis time goes here. These three handle most issues.

4. fillna

Fill missing values with a fixed value or a column statistic.

```
df["discount"] = df["discount"].fillna(0)
df["city"] = df["city"].fillna("Unknown")
```

5. drop_duplicates

Remove exact duplicates, or duplicates on a key while keeping the latest.

```
df = (df.sort_values("updated_at")
        .drop_duplicates("order_id",
keep="last"))
```

6. to_datetime

Convert text dates to real dates, then pull out month, weekday and more.

```
df["date"] = pd.to_datetime(df["date"],
dayfirst=True)
df["month"] = df["date"].dt.to_period("M")
```

@thedataguy16

Filter: 7 and 8

Pick the rows and columns you need.

7. loc

Filter rows by condition and choose columns in one step. Also used to update values.

```
north = df.loc[df["region"] == "North",
               ["order_id", "amount"]]
df.loc[df["amount"] < 0, "amount"] = 0
```

8. query

Readable filtering for several conditions. Use @ to refer to a Python variable.

```
min_amt = 5000
df.query("region == 'North' and amount >
@min_amt")
```

Tip: loc for filtering plus updating. query for long readable conditions.

@thedataguy16

Summarise: 9 and 10

These answer most business questions.

9. groupby

Totals, counts and averages by category. Named aggregations keep output clean.

```
df.groupby("region").agg(
    revenue=("amount", "sum"),
    orders=("order_id", "count"),
    aov=("amount", "mean"))
```

10. value_counts

Frequency of each value. normalize=True gives the share.

```
df["payment_mode"].value_counts(normalize=True)
```

@thedataguy16

Join and reshape: 11 and 12

Combine tables and build report-style views.

11. merge

SQL JOIN in Pandas. validate catches unexpected duplicate keys.

```
orders.merge(customers, on="customer_id",
             how="left", validate="many_to_one")
```

12. pivot_table

Excel pivot in one line. Region by month, revenue in cells.

```
df.pivot_table(index="region", columns="month",
               values="amount", aggfunc="sum",
               fill_value=0, margins=True)
```

validate="many_to_one" raises an error if the right table has duplicate keys. It stops the doubled-rows bug before it happens.

@thedataguy16

All 12 in one analysis

A full monthly revenue report using only these functions.

```
# 1-3 load and inspect
df = pd.read_csv("sales.csv")
df.info()
print(df.isna().sum())
# 4-6 clean
df["discount"] = df["discount"].fillna(0)
df = df.drop_duplicates("order_id")
df["date"] = pd.to_datetime(df["date"])
# 7-8 filter
df.loc[df["amount"] < 0, "amount"] = 0
big = df.query("amount > 5000")
# 9-10 summarise
by_region = df.groupby("region").agg(
    revenue=("amount", "sum"))
print(df["channel"].value_counts(normalize=True))
# 11-12 join and reshape
df = df.merge(customers, on="customer_id",
              how="left")
df["month"] = df["date"].dt.month
report = df.pivot_table(index="region",
    columns="month", values="amount",
    aggfunc="sum", fill_value=0)
```

@thedataguy16

What to skip, and why

You will see these in tutorials. You rarely need them as an analyst.

- **apply with custom functions**

  Slow on large data. Most cases have a built-in vectorised method.

- **iterrows and for loops**

  Hundreds of times slower than column operations.

- **MultiIndex tricks**

  Call reset_index() after groupby and work with normal columns.

- **Learn the 12 deeply instead**

  Know their key arguments (how, keep, aggfunc, normalize) and you can handle most tasks.

@thedataguy16